In [1]:
```
# Ex5: Implement Random Forest and AdaBoost Algorithms and Compare Their
# Name: Tejasvi   USN: 24BTRAO044

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_scor
                             f1_score, confusion_matrix)

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```

In [2]:
```
df = pd.read_csv("/Users/tejashiremath/Downloads/wine_dataset.csv")
X = df.drop(columns=["target", "class_name"])
y = df["target"].rename("target")
class_names = ["class_0", "class_1", "class_2"]

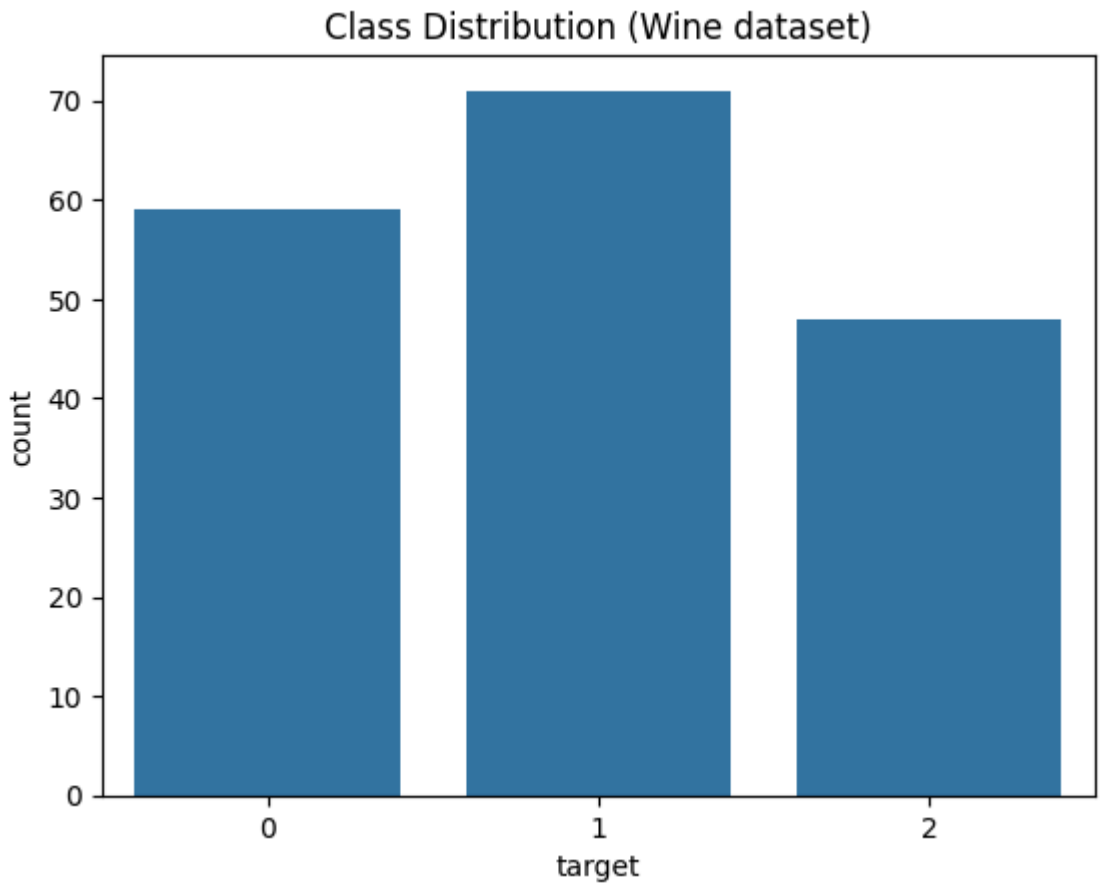
print("Feature matrix shape:", X.shape)
print("Class counts:", y.value_counts().sort_index().tolist())
print("Missing values:", X.isnull().sum().sum())
```

```
Feature matrix shape: (178, 13)
Class counts: [59, 71, 48]
Missing values: 0
```

In [3]:
```
display(X.describe().T[["mean", "std", "min", "max"]].round(2))

sns.countplot(x=y)
plt.title("Class Distribution (Wine dataset)")
plt.show()
```

	mean	std	min	max
alcohol	13.00	0.81	11.03	14.83
malic_acid	2.34	1.12	0.74	5.80
ash	2.37	0.27	1.36	3.23
alcalinity_of_ash	19.49	3.34	10.60	30.00
magnesium	99.74	14.28	70.00	162.00
total_phenols	2.30	0.63	0.98	3.88
flavanoids	2.03	1.00	0.34	5.08
nonflavanoid_phenols	0.36	0.12	0.13	0.66
proanthocyanins	1.59	0.57	0.41	3.58
color_intensity	5.06	2.32	1.28	13.00
hue	0.96	0.23	0.48	1.71
od280/od315_of_diluted_wines	2.61	0.71	1.27	4.00
proline	746.89	314.91	278.00	1680.00



In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_SEED, stratify=y)
print("Training samples:", X_train.shape[0], "| Testing samples:", X_test

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
Training samples: 142 | Testing samples: 36
```

In [5]:
```
param_options = [
    {"n_estimators": 100, "max_depth": 4},
    {"n_estimators": 200, "max_depth": 6},
    {"n_estimators": 300, "max_depth": None},
]
best_score, best_params = -1, None
for params in param_options:
    candidate = RandomForestClassifier(random_state=RANDOM_SEED, **params
    scores = cross_val_score(candidate, X_train, y_train, cv=5)
    print(params, "-> mean CV accuracy:", round(scores.mean(), 4))
    if scores.mean() > best_score:
        best_score, best_params = scores.mean(), params
print("Selected hyperparameters:", best_params)

rf_model = RandomForestClassifier(random_state=RANDOM_SEED, **best_params
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
```

```
{'n_estimators': 100, 'max_depth': 4} -> mean CV accuracy: 0.9862
{'n_estimators': 200, 'max_depth': 6} -> mean CV accuracy: 0.9862
{'n_estimators': 300, 'max_depth': None} -> mean CV accuracy: 0.9862
Selected hyperparameters: {'n_estimators': 100, 'max_depth': 4}
```

In [6]:
```
ada_model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=RANDOM_SEE
    n_estimators=200, learning_rate=1.0, random_state=RANDOM_SEED)
ada_model.fit(X_train, y_train)
ada_pred = ada_model.predict(X_test)

ada_cv_scores = cross_val_score(ada_model, X_train, y_train, cv=5)
print("AdaBoost CV accuracy:", np.round(ada_cv_scores, 4))
print("Mean CV accuracy:", round(ada_cv_scores.mean(), 4))
```

```
AdaBoost CV accuracy: [0.8621 0.9655 1.     0.8214 1.    ]
Mean CV accuracy: 0.9298
```

In [7]:
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
tf.random.set_seed(RANDOM_SEED)

keras_model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense(32, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(len(class_names), activation="softmax"),
])
keras_model.compile(optimizer="adam",
                    loss="sparse_categorical_crossentropy",
                    metrics=["accuracy"])
keras_model.summary()

history = keras_model.fit(X_train_scaled, y_train,
                          validation_split=0.2, epochs=100,
                          batch_size=16, verbose=0)

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch"); plt.ylabel("Accuracy")
```

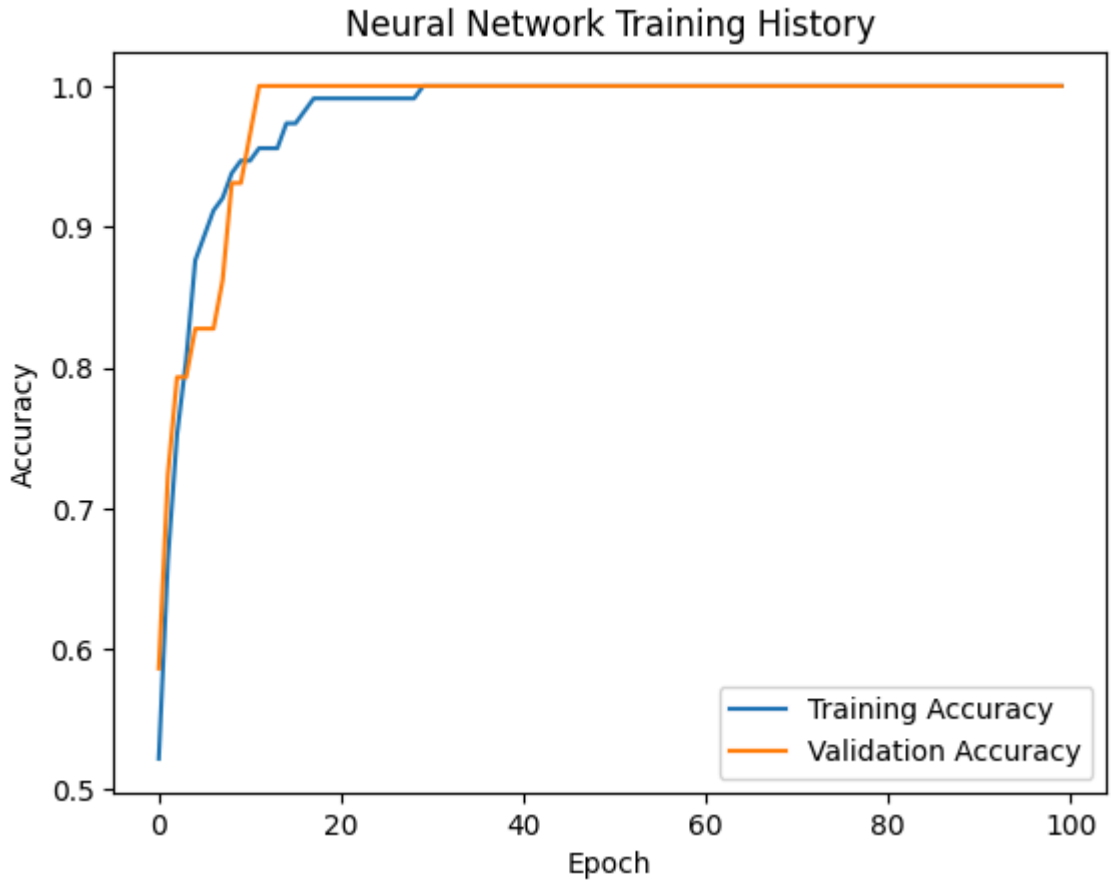
```
plt.title("Neural Network Training History")
plt.legend(); plt.show()

keras_pred = np.argmax(keras_model.predict(X_test_scaled, verbose=0), axi
```

Model: "sequential"

Layer (type)	Output Shape	
dense (Dense)	(None, 32)	
dense_1 (Dense)	(None, 16)	
dense_2 (Dense)	(None, 3)	

Total params: 1,027 (4.01 KB)
Trainable params: 1,027 (4.01 KB)
Non-trainable params: 0 (0.00 B)



In [8]:
```
def evaluate_model(name, y_true, y_pred):
    return {
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, average="macro"),
        "Recall": recall_score(y_true, y_pred, average="macro"),
        "F1-score": f1_score(y_true, y_pred, average="macro"),
    }

results = [
    evaluate_model("Random Forest", y_test, rf_pred),
    evaluate_model("AdaBoost", y_test, ada_pred),
    evaluate_model("Neural Network (TensorFlow/Keras)", y_test, keras_pre
```

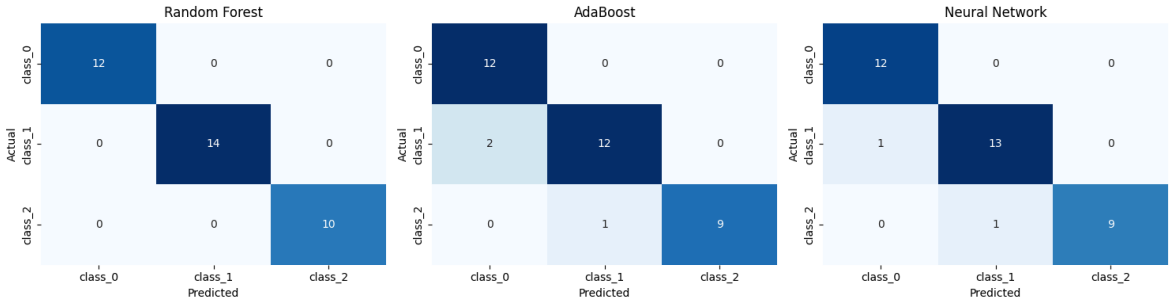
```
]
results_df = pd.DataFrame(results).set_index("Model").round(4)
results_df
```

Out[8]:

Model	Accuracy	Precision	Recall	F1-score
Random Forest	1.0000	1.0000	1.0000	1.0000
AdaBoost	0.9167	0.9267	0.9190	0.9198
Neural Network (TensorFlow/Keras)	0.9444	0.9505	0.9429	0.9453

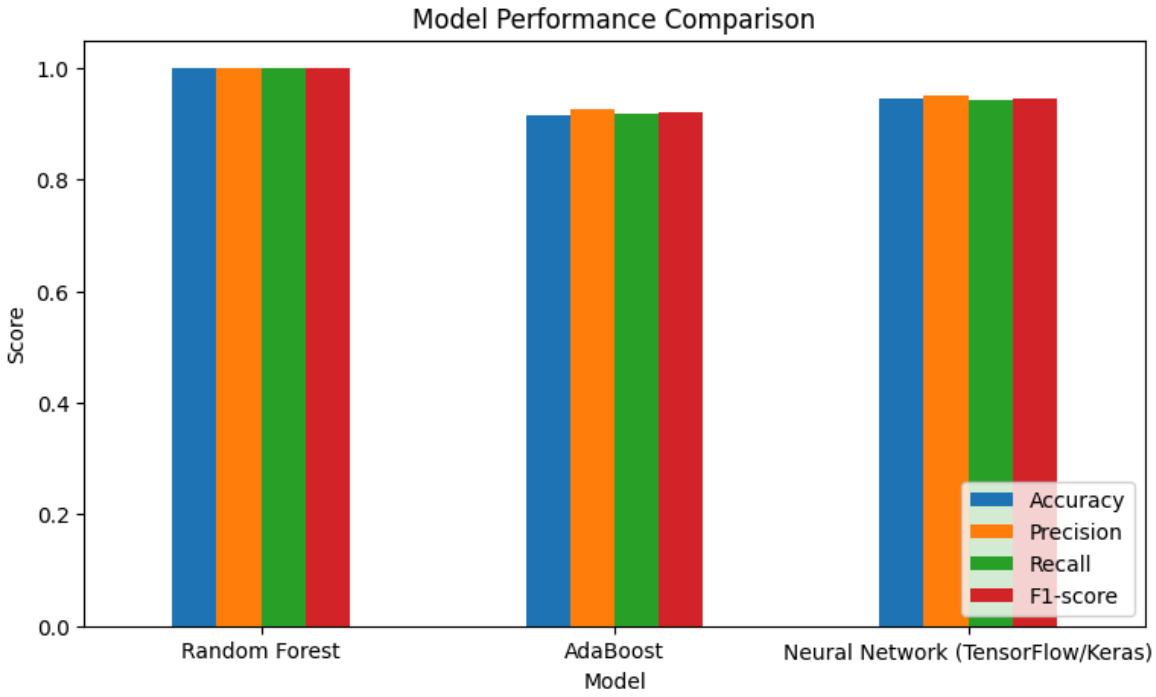
In [9]:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
predictions = {"Random Forest": rf_pred, "AdaBoost": ada_pred, "Neural Ne
for ax, (name, pred) in zip(axes, predictions.items()):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax, cbar=False,
                xticklabels=class_names, yticklabels=class_names)
    ax.set_title(name); ax.set_xlabel("Predicted"); ax.set_ylabel("Actual
plt.tight_layout(); plt.show()
```

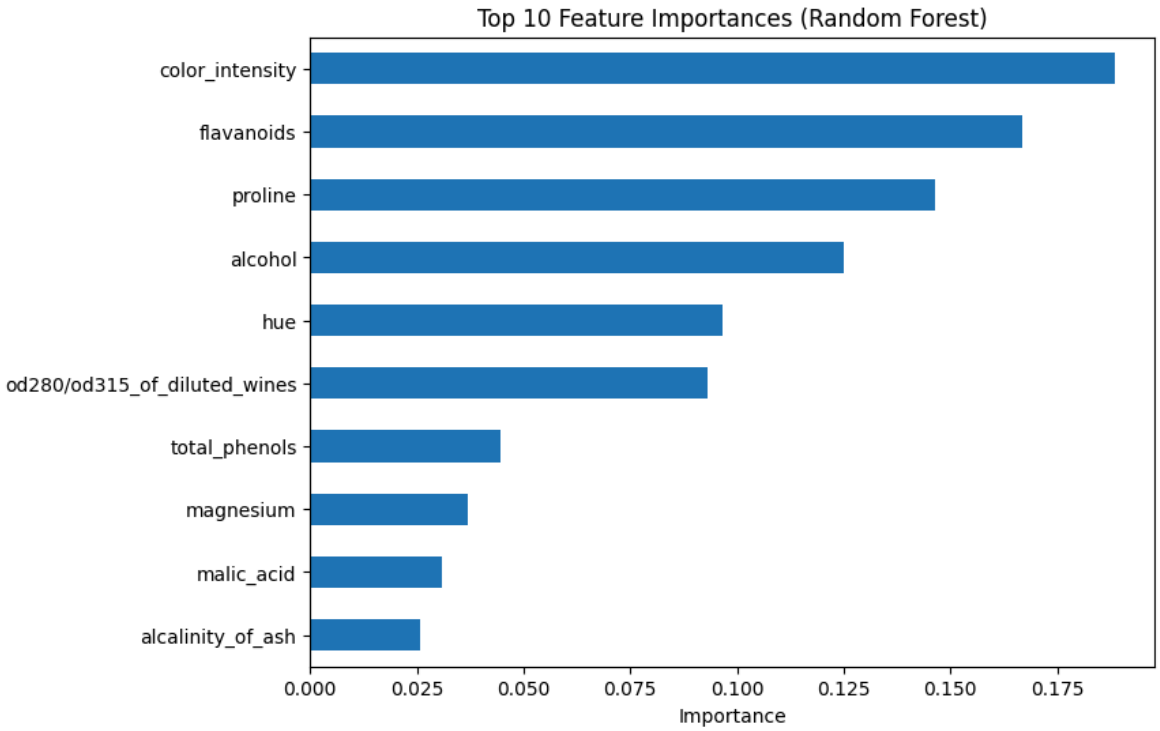


In [10]:

```
results_df.plot(kind="bar", figsize=(9, 5), rot=0)
plt.title("Model Performance Comparison")
plt.ylabel("Score"); plt.ylim(0, 1.05)
plt.legend(loc="lower right"); plt.show()
```



In [11]:
```
importances = pd.Series(rf_model.feature_importances_, index=X.columns)
top_features = importances.sort_values(ascending=False).head(10)
top_features.sort_values().plot(kind="barh", figsize=(8, 6))
plt.title("Top 10 Feature Importances (Random Forest)")
plt.xlabel("Importance"); plt.show()
top_features
```



Out[11]:
```
color_intensity                 0.188447
flavanoids                      0.166997
proline                         0.146483
alcohol                         0.124979
hue                             0.096740
od280/od315_of_diluted_wines    0.093158
total_phenols                   0.044691
magnesium                       0.036984
malic_acid                      0.030844
alcalinity_of_ash               0.025787
dtype: float64
```

In [ ]: